

fastmcp_observability_cal...
Zip Archive

I've started looking into AGENTS.md and how that can be used with Deep Agents. If I pass it as a parameter in the create_deep_agent, what exactly does this do and how does it differ from system_prompt?

I'll ground this in the current Deep Agents behavior and, if useful, peek at your uploaded project to map the explanation to the way your create_deep_agent is being used.

Thought for 37s >

In Deep Agents, you typically pass AGENTS.md through the memory= parameter, not as system_prompt= :

```
agent = create_deep_agent(  
    model=model,  
    tools=tools,  
    system_prompt=APP_SYSTEM_PROMPT,  
    skills=["skills/"],  
    memory=["/AGENTS.md"],  
)
```

What that does: Deep Agents treats AGENTS.md as **persistent memory/context**. Its contents get loaded into the agent's prompt so the agent always knows those project-level conventions, preferences, or rules when it runs. LangChain's docs describe memory files as AGENTS.md files that provide persistent context and are "always injected" when configured. [LangChain Docs](#) +1

The key difference is this:

Area	system_prompt	AGENTS.md via memory=
Main purpose	Defines the agent's core role and behavior	Stores persistent project/user/team conventions

Area	system_prompt	AGENTS.md via memory=
Where it lives	Python code / environment variable	Markdown file
Loading	Always included	Always included when configured
Best for	"You are a helpful assistant... use tools when useful..."	"For this repo, use these commands, naming rules, test rules..."
Can evolve	Usually static unless you change code/config	Can be file-backed and, depending on backend/permissions, updated/persisted
Scope	Agent-level instruction	Memory-like context, can be agent/user/org scoped
Token cost	Always costs tokens	Also always costs tokens, so keep it small

Deep Agents’ final prompt is assembled from multiple pieces. The custom `system_prompt` comes first, then the built-in Deep Agents harness prompt, then planning/tool/filesystem/subagent guidance, and memory/skills sections if configured. The docs list memory as a separate prompt section after the base/todo guidance, not as a replacement for `system_prompt` . [LangChain Docs](#)

For your current app, I’d think of it this way:

```
system_prompt = """
You are the main assistant for this FastMCP observability app.
Use tools when they improve correctness.
Do not invent tool results.
"""

memory = ["AGENTS.md"]

skills = ["skills/"]
```

Use `system_prompt` for the agent’s permanent identity and high-level behavior.

Use `AGENTS.md` for repo/project conventions that should apply to almost every request, for example:

```
# AGENTS.md

## Project overview
This is a FastMCP + Deep Agents prototype with observability callbacks.

## Architecture rules
- Keep the backend structure simple.
- Prefer small, readable files.
- Do not remove observability wiring unless explicitly asked.
- Keep Skills enabled as part of the app.

## Testing
- Run the backend with uvicorn backend.main:app --host 127.0.0.1 --port 8000.
- Use simple prompts like "what is 5+6?" to verify tool tracing.
```

Use `skills/` for task-specific workflows that should not always be loaded. Your math and quote-count skills are good examples. Deep Agents reads only the `SKILL.md` frontmatter at startup, then loads the full skill only when the task matches; that progressive disclosure is the main difference from `AGENTS.md` . [LangChain Docs](#)

So the clean mental model is:

```
system_prompt = who the agent is and how it should generally behave

AGENTS.md = always-relevant project memory / repo rules / conventions

SKILL.md = specialized task instructions loaded only when needed
```

One caution: do **not** put everything into `AGENTS.md` . Since memory is always injected, a large `AGENTS.md` can bloat every request. Put only global, always-relevant rules there, and keep detailed workflows in `SKILL.md` files.

     ...  Sources

how would you create the AGENTS.md for this project?

I'll tailor it to your actual repo rather than give a generic template. I'm going to inspect the project structure and confirm the current Deep Agents conventions so the `AGENTS.md` lines up with how the framework expects it.

I found your app is still the slim FastAPI + Deep Agents + FastMCP version: one `AgentEngine`, native `skills/`, MCP tools through the core gateway, and callback-based observability. The cleanest `AGENTS.md` should reinforce those architectural rules, not duplicate your individual skill instructions.

Thought for 55s >

For **this project**, I'd create `AGENTS.md` as **always-on project guidance** for the agent, not as another skill. So I would keep math behavior in `skills/math/SKILL.md`, quote-count behavior in `skills/quote_count/SKILL.md`, and use `AGENTS.md` for repo-wide rules: architecture, coding style, observability rules, startup commands, and "don't undo these design choices."

Deep Agents supports passing memory files through `memory=[...]`, and the docs specifically show `AGENTS.md` being used that way. Skills are different: they are loaded progressively only when useful for the task, while `memory/AGENTS.md` is broader project context. [LangChain Docs](#) +1

I would put this file here:

```
fastmcp_observability_callback_simple/  
  AGENTS.md  
  README.md  
  backend/  
  gateways/  
  mcp_server/  
  skills/
```

Here's the `AGENTS.md` I'd use:

```
# AGENTS.md  
  
## Project overview
```

This is a small FastAPI + Deep Agents + FastMCP prototype focused on clear architecture and c

The app has one main backend agent that:

- receives chat requests through FastAPI
- maintains simple in-memory session state
- uses Deep Agents native skills from the `skills/` folder
- loads MCP tools through a FastMCP core gateway
- records real LLM and tool activity through LangChain callbacks
- writes one request-level JSONL telemetry row per chat request

Keep the project simple and readable. Prefer explicit, easy-to-follow code over highly abstra

Current architecture

The expected flow is:

```

```text
Client / curl / frontend
 ↓
FastAPI backend
 ↓
ChatSessionStore
 ↓
AgentEngine
 ↓
Deep Agents native skills
 ↓
MCPRuntime
 ↓
Core FastMCP gateway
 ↓
Calculator MCP server + Quote Counter MCP server

```

Important files:

- backend/main.py : FastAPI app, startup wiring, /health , /chat , request telemetry.
- backend/config.py : typed environment settings.
- backend/session\_store.py : in-memory chat sessions and session-level spans.
- backend/agent.py : builds and invokes the Deep Agent.
- backend/mcp\_runtime.py : connects to MCP gateways and loads tools.
- backend/observability/request\_observer.py : callback-based LLM/tool observation.
- backend/observability/models.py : telemetry and metrics models.
- backend/observability/metrics.py : JSONL metrics writer.

- `gateways/core_gateway.py` : mounts built-in MCP servers under namespaces.
- `mcp_server/calculator_server.py` : calculator MCP tools.
- `mcp_server/quote_counter_server.py` : quote-counting MCP tool.
- `skills/math/SKILL.md` : math skill instructions.
- `skills/quote_count/SKILL.md` : quote-count skill instructions.

## Design rules

Preserve these project decisions unless the user explicitly asks to change them:

- Skills are always part of the app.
- Do not add a `SKILLS_ENABLED` flag.
- Do not reintroduce a custom skills registry.
- Use Deep Agents native `skills=[...]`.
- Keep the project small and readable.
- Prefer plain Python service classes for runtime behavior.
- Use Pydantic models for config, API schemas, and telemetry payloads.
- Do not use dataclasses in this project unless explicitly requested.
- Do not estimate LLM calls from message history.
- Do not parse final agent messages to infer tool calls.
- Do not add a tool-copying or tool-wrapping observability layer.
- The source of truth for LLM/tool activity is `RequestObserver`.
- Do not remove OpenTelemetry spans unless explicitly requested.

## Observability rules

The observability design should remain callback-based:

```

AgentEngine.respond()
 ↓
creates one RequestObserver
 ↓
agent.ainvoke(..., config={"callbacks": [observer]})
 ↓
RequestObserver sees real LangChain events
 ↓
metrics are copied into EngineMetrics
 ↓
FastAPI writes one ChatTelemetryEvent JSONL row

```

When changing observability code:

- Keep request-level metrics compact.

- Keep `tools_used` as the actual completed tool names.
- Keep `llm_calls` based on real callback starts.
- Keep tool timing based on `on_tool_start` , `on_tool_end` , and `on_tool_error` .
- Preserve trace IDs when OpenTelemetry is enabled.
- Keep JSONL output human-readable and easy to inspect.

## Skills guidance

This project uses Deep Agents native skills.

Current skills:

```
skills/
 math/
 SKILL.md
 quote_count/
 SKILL.md
```

Use skill files for task-specific procedural instructions.

Do not put detailed math or quote-counting behavior in this file. Keep that logic in the relevant `SKILL.md` .

Use this `AGENTS.md` for global repo behavior only.

## MCP guidance

MCP tools are loaded through the configured gateway URLs.

Default gateway:

```
CORE_GATEWAY_URL=http://127.0.0.1:8090/mcp
```

The core gateway mounts:

- calculator server under the `math` namespace
- quote counter server under the `quote` namespace

Do not bypass the gateway unless the user explicitly asks for a different architecture.

Do not hardcode tool names in the agent when the tools can be discovered from MCP.

## Coding style

Use simple, explicit Python.

Preferred style:

- Python 3.12+
- type hints
- small functions
- clear class responsibilities
- short comments explaining why something exists
- Pydantic BaseModel for structured request, response, config, and telemetry data
- plain classes for runtime services

Avoid:

- unnecessary inheritance
- large framework abstractions
- hidden global state
- custom registries unless clearly needed
- clever callback hacks
- duplicated metrics models
- unnecessary environment toggles

## Runtime commands

Install:

```
conda create -n fastmcp_obs_callback python=3.12 -y
conda activate fastmcp_obs_callback
pip install -e .
cp .env.example .env
```

Run from the project root in four terminals.

Terminal 1:

```
conda activate fastmcp_obs_callback
```

**Bash**

Terminal 2:



```
conda activate fastmcp_obs_callback
```

Bash

Terminal 3:

```
conda activate fastmcp_obs_callback
```

Bash

Terminal 4:

```
conda activate fastmcp_obs_callback
```

Bash

Health check:

Bash

Chat test:

```
curl -X POST http://127.0.0.1:8000/chat \
-H "Content-Type: application/json" \
-d '{"session_id": "demo" "message": "What is 17 + 28? Use a tool "\
```

## Testing expectations

When changing agent, MCP, skill, or observability behavior, test at least:

Bash

Calculator tool test:

```
curl -X POST http://127.0.0.1:8000/chat \
-H "Content-Type: application/json" \
-d '{"session_id": "demo" "message": "What is 17 + 28? Use a tool "\
```

Rach

Quote-count test:

```
curl -X POST http://127.0.0.1:8000/chat \
-H "Content-Type: application/json" \
-d '{"session_id": "quote-test", "message": "How many characters are inside \"hello world\"?"}'
```

Bash

After a chat request, check:

- reply
- tools\_used
- metrics.llm\_calls
- metrics.tool\_calls
- metrics.tool\_total\_duration\_ms
- metrics.tool\_success\_count
- metrics.tool\_error\_count
- the JSONL row in metrics/chat\_metrics.jsonl

## Environment notes

Default local model configuration uses an OpenAI-compatible Ollama endpoint:

```
LLM_MODEL=gpt-oss:20b
LLM_BASE_URL=http://localhost:11434/v1
LLM_API_KEY=ollama
LLM_TEMPERATURE=0.0
```

For OpenAI, update `.env` instead of changing code.

Do not commit real API keys.

## Change discipline

When modifying this project:

1. Keep the existing architecture unless the user asks for a redesign.
2. Update the smallest number of files necessary.
3. Explain which files changed and why.

4. Preserve observability behavior.
5. Preserve native Deep Agents skills.
6. Keep examples simple enough to copy and paste.

Then I'd wire it in with a small change in `backend/agent.py`.

Because Deep Agents treats `memory` paths as files inside its backend/filesystem setup

Example change:

```
```python
from pathlib import Path

from deepagents import create_deep_agent
from deepagents.backends import FileSystemBackend
```



Then inside `AgentEngine.start()` :

Python

Run

```
project_root = Path(__file__).resolve().parents[1]

self.agent = create_deep_agent(
    model=ChatOpenAI(
        model=self.settings.llm_model,
        base_url=self.settings.llm_base_url,
        api_key=self.settings.llm_api_key,
        temperature=self.settings.llm_temperature,
    ),
    tools=tools,
    system_prompt=self.settings.app_system_prompt or DEFAULT_SYSTEM_PROMPT,
    skills=[str(project_root / self.settings.skills_dir)],
    memory=["./AGENTS.md"],
    backend=FileSystemBackend(root_dir=str(project_root)),
)
```

One caution: using a filesystem backend gives the Deep Agent filesystem access through its built-in file tools. Deep Agents has `permissions=` rules for controlling built-in filesystem reads/writes, and the docs note those permissions apply to built-in filesystem tools like `ls`, `read_file`, `write_file`, and `edit_file`, not custom MCP tools.

For your current prototype, I'd start simple with the `AGENTS.md` above. Then, once it's working, decide whether you want it to be **read-only project guidance** or **writable memory**. For this app, I'd lean **read-only** because you want stable architectural rules, not the agent editing its own project instructions during normal chat.

       Sources